

Lab03-Dynamic Programming

CS2312-Algorithm and complexity, Xiaofeng Gao, Spring 2026.

* Contact TA Steven Chen for any questions. Include both your .pdf and .tex files in the uploaded .rar or .zip file.

* Name: Hengtao Wu Student ID: 524031910038 Email: Eternity_w@sjtu.edu.cn

1. (Longest Special Bitonic Subsequence)

Recall the notion of a bitonic sequence discussed in class (e.g., in the context of sorting networks). Suppose you are given a sequence of n integers $\{x_1, x_2, \dots, x_n\}$, where $x_i \in \mathbb{Z}$.

We define a *special bitonic subsequence* as a subsequence in which every monotonically increasing or decreasing segment has length exactly 2.

$$\text{sgn}(x) = \begin{cases} -1 & x < 0 \\ 0 & x = 0 \\ 1 & x > 0 \end{cases} \quad \text{Ex 1: } \langle 6, 9, 4, 2, 3, 5 \rangle \text{ has multiple valid optimal subsequences: } \langle 6, 9, 2, 3 \rangle, \langle 6, 9, 2, 5 \rangle, \text{ and } \langle 6, 9, 4, 5 \rangle.$$

- (a) Express your DP algorithm using the standard recurrence template.
- (b) Design a DP algorithm that computes the length of the longest special bitonic subsequence in $O(n)$ time. Provide pseudocode and briefly justify your running time.

(Hint: A useful helper is the *signum function*, defined as above)

Solution. (a) We define $Up[i]$ as the length of the longest valid special bitonic subsequence ending with an *ascent* at position i (i.e. $x_{i-1} < x_i$), and $Down[i]$ as the length of the longest such subsequence ending with a *descent* at position i (i.e. $x_{i-1} > x_i$).

Because every monotone segment must have length *exactly* 2, each transition appends exactly one new element by reversing direction; hence we only need to compare x_i with its immediate predecessor x_{i-1} .

Base case:

$$Up[1] = Down[1] = 1.$$

Recurrence:

$$Up[i] = \begin{cases} Down[i-1] + 1 & \text{if } \text{sgn}(x_i - x_{i-1}) = 1, \\ Up[i-1] & \text{otherwise.} \end{cases}$$
$$Down[i] = \begin{cases} Up[i-1] + 1 & \text{if } \text{sgn}(x_i - x_{i-1}) = -1, \\ Down[i-1] & \text{otherwise.} \end{cases}$$

Goal: $\max(Up[n], Down[n])$.

- (b) The pseudocode is given below.

Note on update order: the two branches are mutually exclusive (only one of $s = 1$ or $s = -1$ can hold at a time), so updating UP and $DOWN$ in place does not cause stale-value issues.

Time complexity: The loop runs $n - 1$ times and each does $O(1)$, giving $O(n)$ overall.

Space complexity: Only two scalar variables are maintained, giving $O(1)$ auxiliary space.

Algorithm 1: Longest Special Bitonic Subsequence ($O(n)$ time, $O(1)$ space)

Input: An array of n integers $X = \{x_1, x_2, \dots, x_n\}$

Output: Length of the longest special bitonic subsequence

```
1  $UP \leftarrow 1$ ;
2  $DOWN \leftarrow 1$ ;
3 for  $i \leftarrow 2$  to  $n$  do
4    $s \leftarrow \text{sgn}(x_i - x_{i-1})$ ;
5   if  $s = 1$  then
6      $UP \leftarrow DOWN + 1$ ;
7   else if  $s = -1$  then
8      $DOWN \leftarrow UP + 1$ ;
9    $// s = 0$ : equal elements, no update needed
9 return  $\max(UP, DOWN)$ ;
```

□

2. (Windy World DP)

An agent is placed in a $m \times n$ grid. The start state is $(0, 0)$ and the goal state is $(m-1, n-1)$. The agent cannot move outside the grid. At each step, the agent may take one of the following actions:

$$a_1 : (i, j) \rightarrow (i, j+1) \quad (\text{right}), \quad a_2 : (i, j) \rightarrow (i+1, j) \quad (\text{down}).$$

In addition, the agent is pushed 1 cell to the right whenever it enters a cell in the “windy” rows. The row numbers of the “windy” rows are stored in an array w . The task is to compute the number of distinct paths from the start to the goal.

- (a) Design a recursive algorithm to compute the number of valid paths. Provide pseudocode.
- (b) Improve your solution using memoization or tabulation. Compare it with the recursive approach in terms of time and space complexity.

Solution. (a) The pseudo code is shown as below. We assume that the value of w in windy’s row is 1, otherwise 0.

Algorithm 2: Windy World Valid Paths (Recursive)

Input: Current row i , current column j , wind array w

Output: Number of valid paths reaching (i, j)

```
1 if  $i < 0$  or  $j < 0$  then
2   return 0;
3 if  $i = 0$  and  $j = 0$  then
4   return 1;
5 if  $w[i] = 1$  then
6    $// (i, 0)$  is unreachable in a windy row
7   if  $j = 0$  then return 0;
8   return  $WW(i-1, j-1, w) + WW(i, j-2, w)$ ;
9 else
10  return  $WW(i-1, j, w) + WW(i, j-1, w)$ ;
```

Usage: call $WW(m-1, n-1, w)$.

(b) **Improvement Strategy:**

We improve the algorithm using **Tabulation**. Instead of recursively solving from the goal state and redundantly calculating overlapping subproblems, we iteratively build the solution from the start state $(0, 0)$ to the goal state $(m - 1, n - 1)$ using a 2D array $path[m][n]$.

Complexity Comparison:

- **The Recursive Approach:**

- **Time Complexity:** $\mathcal{O}(2^{m+n})$. At each step, the algorithm branches into two separate recursive calls. Because it does not store previously computed results, it recalculates the same overlapping subproblems multiple times, leading to exponential time complexity.
- **Space Complexity:** $\mathcal{O}(m + n)$. This is determined by the maximum depth of the recursion call stack, which corresponds to the longest possible path from the goal $(m - 1, n - 1)$ back to the start $(0, 0)$.

- **The Tabulation Approach (DP):**

- **Time Complexity:** $\mathcal{O}(m \cdot n)$. We use two nested loops to iterate through the entire $m \times n$ grid exactly once. At each cell, the state transition takes $\mathcal{O}(1)$ constant time.
- **Space Complexity:** $\mathcal{O}(m \cdot n)$. The algorithm requires an explicit 2D array of size $m \times n$ to store the number of valid paths. (*Note: Space complexity can be further optimized to $\mathcal{O}(n)$ using a 1D rolling array.*)

The optimized pseudocode is shown as below :

Algorithm 3: Windy World Valid Paths (Tabulation)

Input: Grid dimensions $m \times n$, wind array w

Output: Total number of valid paths to $(m - 1, n - 1)$

```
1 Initialize  $path[m][n]$  with all 0s;
2  $path[0][0] \leftarrow 1$ ;
3 for  $i \leftarrow 0$  to  $m - 1$  do
4   for  $j \leftarrow 0$  to  $n - 1$  do
5     if  $i = 0$  and  $j = 0$  then continue;
6     if  $w[i] = 1$  then
7       if  $j = 0$  then  $path[i][j] \leftarrow 0$ ;
8       else
9          $v_1 \leftarrow (i \geq 1 \text{ and } j \geq 1) ? path[i - 1][j - 1] : 0$ ;
10         $v_2 \leftarrow (j \geq 2) ? path[i][j - 2] : 0$ ;
11         $path[i][j] \leftarrow v_1 + v_2$ ;
12      else
13         $v_1 \leftarrow (i \geq 1) ? path[i - 1][j] : 0$ ;
14         $v_2 \leftarrow (j \geq 1) ? path[i][j - 1] : 0$ ;
15         $path[i][j] \leftarrow v_1 + v_2$ ;
16 return  $path[m - 1][n - 1]$ ;
```

□

3. (LP)

A grass carp farm must determine a daily feeding plan that minimizes cost while satisfying nutritional requirements. Each day, the fish must receive at least the following amounts of nutrients:

Nutrient type	Nutrient A	Nutrient B
Required (% of total biomass W)	10%	6%

Two types of feed are available:

Feed type	Cost (\$/kg)	Nutrient A (per kg)	Nutrient B (per kg)
Water plants	5	0.6	0.2
Commercial fish feed	10	0.2	0.7

Determine a feeding strategy that meets the nutritional requirements at minimum cost.

- Formulate the problem as a linear program.
- Solve the linear program. Express the minimum daily cost as a function of the total biomass W .

Solution. (a) **Linear Program Formulation**

Let x_1 be the daily amount of water plants used (in kg), and x_2 be the daily amount of commercial fish feed used (in kg).

$$\begin{aligned}
 \min \quad & Z = 5x_1 + 10x_2 \\
 \text{s.t.} \quad & 0.6x_1 + 0.2x_2 \geq 0.10W \\
 & 0.2x_1 + 0.7x_2 \geq 0.06W \\
 & x_1, x_2 \geq 0
 \end{aligned}$$

(b) **Solving the Linear Program**

The feasible region is the intersection of two half-planes (above each constraint line) in the first quadrant, which is a convex set unbounded toward the upper right. Since the objective function increases in that direction, the minimum occurs at the unique finite vertex, i.e. the intersection of the two active constraints:

$$0.6x_1 + 0.2x_2 = 0.10W \quad (1)$$

$$0.2x_1 + 0.7x_2 = 0.06W \quad (2)$$

From (1): $x_2 = 0.5W - 3x_1$. Substituting into (2):

$$0.2x_1 + 0.7(0.5W - 3x_1) = 0.06W \implies -1.9x_1 = -0.29W \implies x_1^* = \frac{29}{190}W$$

$$x_2^* = 0.5W - 3 \cdot \frac{29}{190}W = \frac{4}{95}W$$

The minimum daily cost is:

$$Z^* = 5 \cdot \frac{29}{190}W + 10 \cdot \frac{4}{95}W = \frac{145}{190}W + \frac{80}{190}W = \frac{45}{38}W$$

□